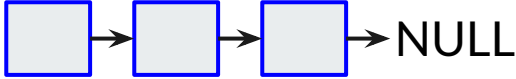




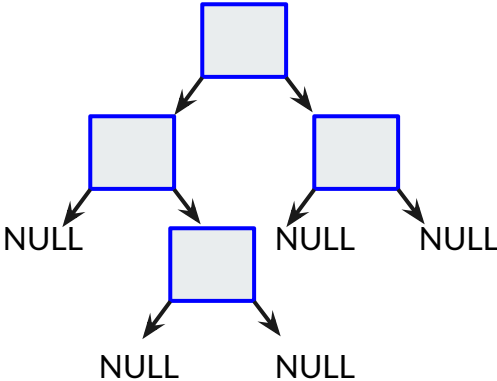
ED1 08: Grafos

Fabio Irigon Pereira

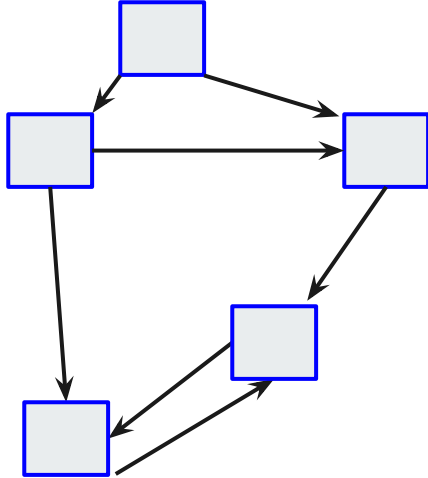
Lista



Árvore



Grafo



Conceitos



Grafo:

Um **grafo** é um conjunto de **vértices (nós)** e **arestas (ligações)**.

Tipos:

- **Direcionado (digrafo):** Arestas têm direção.
- **Não-direcionado:** Arestas são bidirecionais.
- **Ponderado:** Arestas possuem peso.
- **Não-ponderado:** Arestas não têm peso.

Aplicações de grafos



- Redes sociais (conexões entre usuários)
- Mapas e rotas (ruas como arestas)
- Redes de computadores
- Planejamento de tarefas (grafos direcionados acíclicos)
- Jogos (mapas, movimentação, IA)

Representação



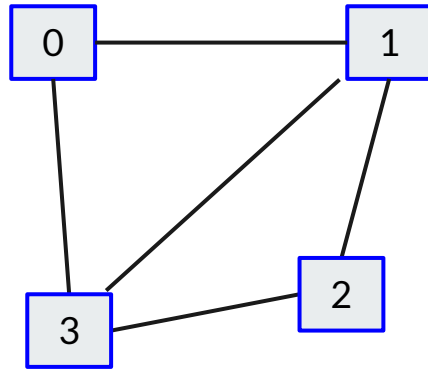
- A Matriz de Adjacência
- Lista de Adjacência

Representação

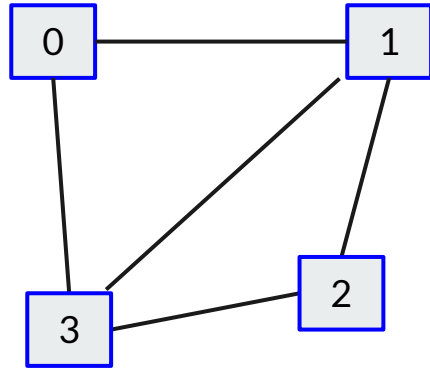


- A Matriz de Adjacência
 - É uma matriz $n \times n$ (onde n é o número de vértices).
 - $\text{mat}[i][j] = 1$ (ou peso) indica aresta de i para j .
 - Boa para grafos densos (muitos vértices ligados).
 - Consome muita mem para grafos esparsos.

Como ficaria a Matriz de Adjacência?



Como ficaria a Matriz de Adjacência?



0	1	0	1
1	0	1	1
0	1	0	1
1	1	1	0

Matriz de adjacência em C

```
#include <stdio.h>
```

```
#define MAX 100
```

```
void inicializarMatriz(int matriz[MAX][MAX], int n) {  
    for (int i = 0; i < n; i++)  
        for (int j = 0; j < n; j++)  
            matriz[i][j] = 0;  
}
```

```
void adicionarAresta(int matriz[MAX][MAX], int u, int v) {  
    matriz[u][v] = 1;  
    matriz[v][u] = 1; // Para grafos não direcionados  
}
```

```
void imprimirMatriz(int matriz[MAX][MAX], int n) {  
    printf("Matriz de Adjacência:\n");  
    for (int i = 0; i < n; i++) {  
        for (int j = 0; j < n; j++)  
            printf("%d ", matriz[i][j]);  
        printf("\n");  
    }  
}
```

```
int main() {  
    int n = 5;  
    int adj[MAX][MAX];  
    inicializarMatriz(adj, n);
```

```
    adicionarAresta(adj, 0, 1);  
    adicionarAresta(adj, 0, 2);  
    adicionarAresta(adj, 1, 3);
```

```
    imprimirMatriz(adj, n);  
    return 0;
```

```
}
```

Lista de Adjacência



Cada vértice possui uma lista ligada com seus vizinhos.

Vantagens:

- Eficiência de memória em grafos esparsos.
- Permite armazenar pesos facilmente.

Desvantagens:

- Acesso a adjacência é $O(\text{grau}(v))$.

Exmplo Lista adjacência em C

```
#include <stdio.h>
#include <stdlib.h>

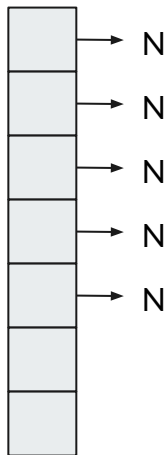
#define MAX 100

typedef struct No {
    int vertice;
    struct No* prox;
} No;

typedef struct {
    No* lista[MAX];
    int nVertices;
} Grafo;

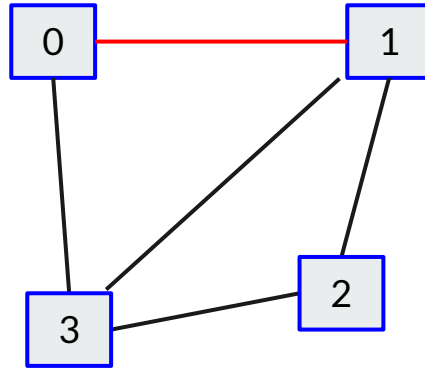
void inicializarGrafo(Grafo* g, int n) {
    g->nVertices = n;
    for (int i = 0; i < n; i++)
        g->lista[i] = NULL;
}
```

Grafo inicializado com 5 elementos

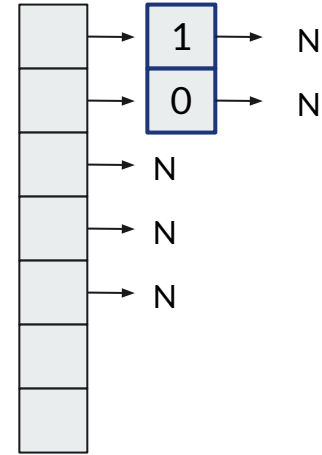


Exmplo Lista adjacência em C

```
void adicionarAresta(Grafo* g, int u, int v) {  
    No* n = novoNo(v);  
    n->prox = g->lista[u];  
    g->lista[u] = n;  
  
    n = novoNo(u); // Para grafo não direcionado  
    n->prox = g->lista[v];  
    g->lista[v] = n;  
}
```

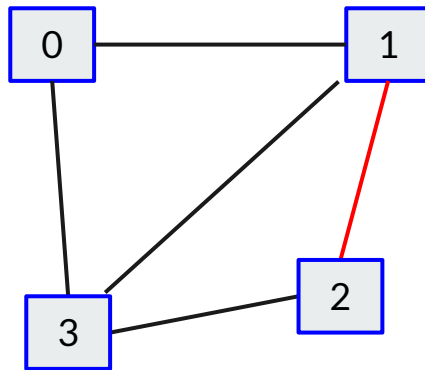


Adicionando primeira aresta

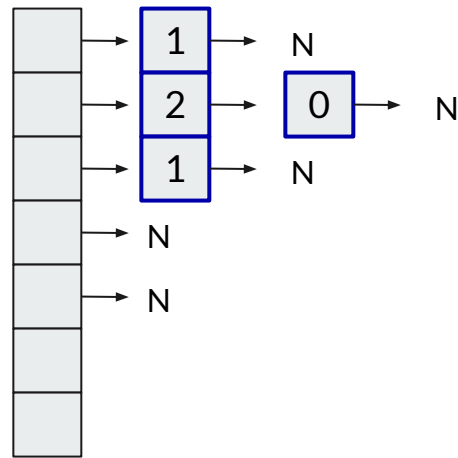


Exmplo Lista adjacência em C

```
void adicionarAresta(Grafo* g, int u, int v) {  
    No* n = novoNo(v);  
    n->prox = g->lista[u];  
    g->lista[u] = n;  
  
    n = novoNo(u); // Para grafo não direcionado  
    n->prox = g->lista[v];  
    g->lista[v] = n;  
}
```



Adicionando segunda aresta



Código Lista adjacência em C

```
#include <stdio.h>
#include <stdlib.h>

#define MAX 100

typedef struct No {
    int vertice;
    struct No* prox;
} No;

typedef struct {
    No* lista[MAX];
    int nVertices;
} Grafo;

No* novoNo(int v) {
    No* n = malloc(sizeof(No));
    n->vertice = v;
    n->prox = NULL;
    return n;
}
```

```
void inicializarGrafo(Grafo* g, int n) {
    g->nVertices = n;
    for (int i = 0; i < n; i++)
        g->lista[i] = NULL;
}

void adicionarAresta(Grafo* g, int u, int v) {
    No* n = novoNo(v);
    n->prox = g->lista[u];
    g->lista[u] = n;

    n = novoNo(u); // Para grafo não direcionado
    n->prox = g->lista[v];
    g->lista[v] = n;
}
```

```
void imprimirLista(Grafo* g) {
    for (int i = 0; i < g->nVertices; i++) {
        printf("Vértice %d:", i);
        No* temp = g->lista[i];
        while (temp) {
            printf(" %d", temp->vertice);
            temp = temp->prox;
        }
        printf("\n");
    }
}

int main() {
    Grafo g;
    inicializarGrafo(&g, 5);
    adicionarAresta(&g, 0, 1);
    adicionarAresta(&g, 0, 2);
    adicionarAresta(&g, 1, 3);
    imprimirLista(&g);
    return 0;
}
```

Exercícios



- Represente um grafo não-direcionado com 5 vértices em matriz e lista.
- Implemente a verificação de vizinhança entre dois vértices.
- Conte o grau de cada vértice usando ambas as representações.
- Altere sua lista de adjacência para lidar com grafos direcionados e ponderados.
- Implemente uma função que imprima todos os vizinhos de um vértice.

Percurso em Gráficos BFS e DFS

Conceitos Fundamentais



◆ Busca em Largura (BFS)

- Explora vértices por **níveis** (primeiro os mais próximos).
- Utiliza **fila (FIFO)**
- Útil para encontrar caminhos mínimos em grafos não ponderados.

◆ Busca em Profundidade (DFS)

- Explora cada caminho até o fim antes de retroceder.
- Utiliza **recursão** ou **pilha**.
- Útil para detectar ciclos, componentes conexas, ordenação topológica.

Busca em Largura (BFS)

```
void BFS(Grafo* g, int inicio) {
    int fila[MAX], frente = 0, tras = 0;

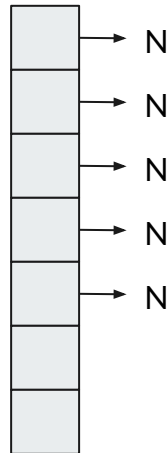
    g->visitado[inicio] = 1;
    fila[tras++] = inicio;
    printf("BFS a partir do vértice%d: ", inicio);

    while (frente < tras) {
        int atual = fila[frente++];
        printf("%d ", atual);

        No* temp = g->lista[atual];
        while (temp != NULL) {
            int v = temp->vertice;
            if (!g->visitado[v]) {
                g->visitado[v] = 1;
                fila[tras++] = v;
            }
            temp = temp->prox;
        }
    }

    printf("\n");
}
```

Grafo



visitado

0	0	0	0	0	
---	---	---	---	---	--

Busca em Largura (BFS)

```
void BFS(Grafo* g, int inicio) {
    int fila[MAX], frente = 0, tras = 0;

    g->visitado[inicio] = 1;
    fila[tras++] = inicio;
    printf("BFS a partir do vértice %d: ", inicio);

    while (frente < tras) {
        int atual = fila[frente++];
        printf("%d ", atual);

        No* temp = g->lista[atual];
        while (temp != NULL) {
            int v = temp->vertice;
            if (!g->visitado[v]) {
                g->visitado[v] = 1;
                fila[tras++] = v;
            }
            temp = temp->prox;
        }
    }

    printf("\n");
}
```

Enquanto tiver elementos na fila:

- Retira o primeiro elemento da fila
- Adiciona os vizinhos à fila.

Busca em Profundidade (DFS)



Algoritmo recursivo (poderia usar pilha)

```
void DFS(Grafo* g, int v) {
    g->visitado[v] = 1;
    printf("%d ", v);

    No* temp = g->lista[v];
    while (temp != NULL) {
        int adj = temp->vertice;
        if (!g->visitado[adj]) {
            DFS(g, adj);
        }
        temp = temp->prox;
    }
}
```

- Marca o nó inicial como visitado.
- Para cada vizinho não visitado:
 - chama a função passando o vizinho como nó inicial.

Exercício:



Crie o grafo com as conexões:

0-1, 0-2, 1-3, 2-4, 3-5, 4-6, 5-7

Percorra os grafos usando BFS e DFS.

Caminho mínimo (Dijkstra)

Dijkstra - caminho mínimo:



O **algoritmo de Dijkstra** encontra o **caminho de menor custo** de um **vértice de origem** até todos os demais vértices de um **grafo com pesos não-negativos**.

- Funciona de maneira gulosa (greedy).
- Mantém um conjunto de vértices já visitados.
- A cada passo, escolhe o vértice com menor custo acumulado que ainda não foi visitado.
- Atualiza as distâncias dos vizinhos.

Desenhar o grafo:



Exemplo de grafo:

Vértices: 0, 1, 2, 3

Arestas:

$0 \rightarrow 1$ (peso 4)

$0 \rightarrow 2$ (peso 1)

$1 \rightarrow 3$ (peso 1)

$2 \rightarrow 1$ (peso 2)

$2 \rightarrow 3$ (peso 5)

Desenhe o grafo e responda:
Qual o menor caminho entre 0 e 3?

Menor distância



```
int menorDistancia(int dist[], int visitado[], int n) {  
    int min = INF, minIndex = -1;  
    for (int i = 0; i < n; i++) {  
        if (!visitado[i] && dist[i] < min) {  
            min = dist[i];  
            minIndex = i;  
        }  
    }  
    return minIndex;  
}
```

Código completo

```
void dijkstra(Grafo* g, int origem) {
    int dist[MAX];
    int visitado[MAX];

    for (int i = 0; i < g->nVertices;
i++) {
        dist[i] = INF;
        visitado[i] = 0;
    }

    dist[origem] = 0;
```

```
        for (int i = 0; i < g->nVertices - 1;
i++) {
            int u = menorDistancia(dist,
visitado, g->nVertices);
            if (u == -1) break;
            visitado[u] = 1;

            No* temp = g->lista[u];
            while (temp != NULL) {
                int v = temp->vertice;
                int peso = temp->peso;

                if (!visitado[v] && dist[u] +
peso < dist[v]) {
                    dist[v] = dist[u] + peso;
                }

                temp = temp->prox;
            }
        }
```

```
        printf("Distâncias a partir do
vértice %d:\n", origem);
        for (int i = 0; i < g->nVertices;
i++) {
            printf("Para %d: %d\n", i,
dist[i]);
```

```
        printf("Distâncias a partir do
vértice %d:\n", origem);
        for (int i = 0; i < g->nVertices;
i++) {
            printf("Para %d: %d\n", i,
dist[i]);
        }
    }
```

dijkstra - A



```
void dijkstra(Grafo* g, int origem) {  
    int dist[MAX];  
    int visitado[MAX];  
  
    for (int i = 0; i < g->nVertices; i++) {  
        dist[i] = INF;  
        visitado[i] = 0;  
    }  
  
    dist[origem] = 0;
```

...

dist

0	i	i	i		
---	---	---	---	--	--

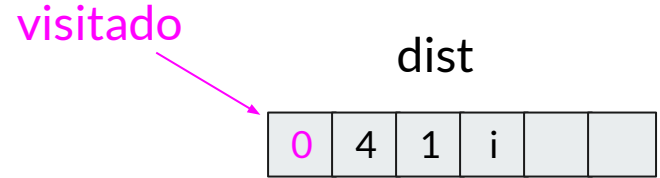
dijkstra - B

```
for (int i = 0; i < g->nVertices - 1; i++) {  
    int u = menorDistancia(dist, visitado, g->nVertices);  
    if (u == -1) break;  
    visitado[u] = 1;  
  
    No* temp = g->lista[u];  
    while (temp != NULL) {  
        int v = temp->vertice;  
        int peso = temp->peso;  
  
        if (dist[u] + peso < dist[v]) {  
            dist[v] = dist[u] + peso;  
        }  
  
        temp = temp->prox;  
    }  
}
```

...

$0 \rightarrow 1$ (peso 4)

$0 \rightarrow 2$ (peso 1)



Demonstração do algoritmo para o grafo desenhado.

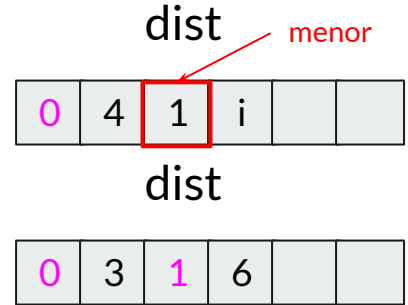
dijkstra - C

```
for (int i = 0; i < g->nVertices - 1; i++) {  
    int u = menorDistancia(dist, visitado, g->nVertices);  
    if (u == -1) break;  
    visitado[u] = 1;  
  
    No* temp = g->lista[u];  
    while (temp != NULL) {  
        int v = temp->vertice;  
        int peso = temp->peso;  
  
        if (dist[u] + peso < dist[v]) {  
            dist[v] = dist[u] + peso;  
        }  
  
        temp = temp->prox;  
    }  
}
```

...

2 → 1 (peso 2)

2 → 3 (peso 5)



Nós 0 e 2 marcados como visitados.
Distâncias para origem atualizadas.

dijkstra - D

```
for (int i = 0; i < g->nVertices - 1; i++) {  
    int u = menorDistancia(dist, visitado, g->nVertices);  
    if (u == -1) break;  
    visitado[u] = 1;
```

```
    No* temp = g->lista[u];  
    while (temp != NULL) {  
        int v = temp->vertice;  
        int peso = temp->peso;  
  
        if (dist[u] + peso < dist[v]) {  
            dist[v] = dist[u] + peso;  
        }  
    }
```

```
    temp = temp->prox;
```

```
}
```

...

1 → 3 (peso 1)

dist

0	4	1	i		
---	---	---	---	--	--

dist

0	3	1	6		
---	---	---	---	--	--

menor

dist

0	3	1	4		
---	---	---	---	--	--

Desenhe o grafo:

A - C: 3

A - F: 2

F - C: 2

C - E: 1

F - E: 3

C - D: 4

F - G: 5

D - B: 1

G - B: 2

F - B: 6

E - B: 2

Como ficaria o array visitado e o array distância em cada passo do dijkstra começando pelo nó 'A'?

visitado

1	0	0	0	0	0	0
---	---	---	---	---	---	---

distância

0	∞	∞	∞	∞	∞	∞
---	----------	----------	----------	----------	----------	----------

resposta: https://youtu.be/EFg3u_E6eHU?si=wclG9XUH1LvMfTcH